```
WebApplicationBuilder builder = WebApplication.CreateBuilder(args);
WebApplication app = builder.Build();

app.MapGet("/products/search",
    (int[] id) => $"Received {id.Length} ids");     ❶

app.Run();
```

❶ The array will bind to multiple instances of id in the query string.

If you're anything like me, the fact that the `int[]` handler parameter in listing 7.4 is called `id` and not `ids` will really bug you. Unfortunately, you have to use `id` here so that the parameter binds correctly to a query string like `?id=123&id=456`. If you renamed it `ids`, the query string would need to be `?ids=123&ids=456`.

Luckily, you have another option. You can control the name of the target that a handler parameter binds to by using the `[FromQuery]` and `[FromRoute]` attributes, similar to the way you use `[FromHeader]`. For this example, you can have the best of both worlds by renaming the handler parameter `ids` and adding the `[FromQuery]` attribute:

```
app.MapGet("/products/search",
    ([FromQuery(Name = "id")] int[] ids) => $"Received {ids.Length} ids");
```

Now you can sleep easy. The handler parameter has a better name, but it still binds to the query string `?id=123&id=456` correctly.

**TIP** You can bind array parameters to multiple header values in the same way that you do for as query string values, using the `[FromHeader]` attribute.